Artificial Intelligence

BS (CS) Spring 2026

# Informed Searches




## Learning Objectives:

1. Informed Search
2. A*Algorithm
3. Greedy Best-First Search

# Practice Tasks

# Questions

**Q1.** Design and implement a Python program to solve the 8-puzzle problem using the A* search algorithm. The 8-puzzle consists of a 3×3 board containing eight numbered tiles (1–8) and one blank space represented by 0. A tile may move into the blank space if it is directly adjacent to it in one of the four directions: up, down, left, or right. The program should take the following initial state as input:

**Initial State**

| | | |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 5 | 6 |
| 4 | 7 | 8 |

**Goal State**

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 0 |

Based on problem analysis, select and implement an appropriate admissible heuristic function such as **Manhattan Distance** or **Euclidean Distance**. The Manhattan Distance calculates the sum of the absolute row and column differences between each tile's current position and its goal position, while the Euclidean Distance calculates the straight-line distance between the current and goal positions of each tile. The A* evaluation function should be defined as $f(n) = g(n) + h(n)$, where $g(n)$represents the path cost from the initial state to the current state and $h(n)$represents the heuristic estimate to the goal.

The program should generate all valid successor states, apply the legal moves, and display the sequence of moves along with the total cost required to reach the goal state. Additionally, justify which heuristic is more suitable for the 8-puzzle problem and explain your choice based on efficiency and optimality considerations.

**Q2.** Design and implement a Python program to solve the Maze Navigation Problem using the A* search algorithm. Consider a rescue robot operating in a disaster-affected building represented as a 2D grid. Each cell in the grid can either be a free space (0) or an obstacle (1). The robot can move only in four directions: up, down, left, and right. Diagonal movement is not allowed. Each valid move has a uniform cost of 1.

The maze is defined as follows:

| | | | | |
|---|---|---|---|---|
| **S** | **0** | **1** | **0** | **0** |
| **0** | 0 | 1 | 0 | 1 |
| **1** | 0 | 0 | 0 | 0 |
| **0** | 1 | 1 | 1 | 0 |
| **0** | 0 | 0 | 1 | G |

Where:

- `S` represents the start position (0,0)
- `G` represents the goal position (4,4)
- `0` represents a traversable cell
- `1` represents an obstacle

You are required to:

1. Formulate the problem by clearly defining the state space, initial state, goal state, and successor function.
2. Implement the A* evaluation function $f(n) = g(n) + h(n)$, where $g(n)$is the path cost from the start node to the current node and $h(n)$is a heuristic estimate of the remaining cost to the goal.
3. Select an appropriate admissible heuristic function (Manhattan Distance or Euclidean Distance) based on the movement constraints, and justify your choice.
4. Determine and display the optimal path from the start state to the goal state.
5. Report the total path cost and the number of nodes expanded during the search.

Provide a brief analysis of the impact of the chosen heuristic on the efficiency and optimality of the solution.

**Q3.** In a smart city, emergency vehicles such as ambulances must be routed efficiently from a hospital to an accident site. The city is modeled as a **weighted graph**, where intersections are represented as **nodes** and roads connecting intersections are represented as **edges** with associated travel costs, reflecting distance, travel time, or congestion.

Given the following road network:

- Intersections: A, B, C, D, E, F, G
- Roads and travel costs:

```
# Graph
graph = {
    'A': [('B', 4), ('C', 3)],
    'B': [('D', 5), ('E', 12)],
    'C': [('F', 7)],
    'D': [('G', 9)],
    'E': [('G', 4)],
    'F': [('G', 2)],
    'G': []  # Goal node
}

# Heuristic values
heuristic = {
    'A': 10,
    'B': 8,
    'C': 6,
    'D': 5,
    'E': 3,
    'F': 1,
    'G': 0
}
```

The ambulance starts at **Intersection A**, and the goal is **Intersection G**. The heuristic values $h(n)$, representing the estimated straight-line distance to the goal, are provided as follows:

- h(A) = 10, h(B) = 8, h(C) = 6, h(D) = 5, h(E) = 3, h(F) = 1, h(G) = 0

**Tasks:**

1. Formulate the problem by defining the **state space**, **initial state**, **goal state**, and **successor function**.
2. Implement the A* search algorithm using the evaluation function:

$$f(n) = g(n) + h(n)$$

where $g(n)$is the path cost from the start node to the current node, and $h(n)$is the heuristic estimate to the goal.

3. Generate and display the **node expansion sequence** and corresponding **f(n) values**.
4. Determine the **optimal path** from Intersection A to Intersection G and calculate the **total travel cost**.
5. Explain how the choice of heuristic affects the **efficiency and optimality** of the solution.

**Q4.** In a remote smart village, a relief team is tasked with distributing clean drinking water to households during a temporary shortage. The team has two water containers mounted on a mobile supply trolley: **Container A**, which can hold up to 4 liters, and **Container B**, which can hold up to 3 liters. Both containers are initially empty and have no measurement markings.

The village authorities require that exactly **2 liters of water must be delivered in Container A** to a household for safe consumption. The team can only use the following operations:

1. Fill a container completely from the main water supply.
2. Empty a container completely.
3. Pour water from one container into the other until either the source container is empty or the target container is full.

Because the water supply is limited, the team must carefully plan the sequence of operations to achieve exactly 2 liters in Container A without wasting water.

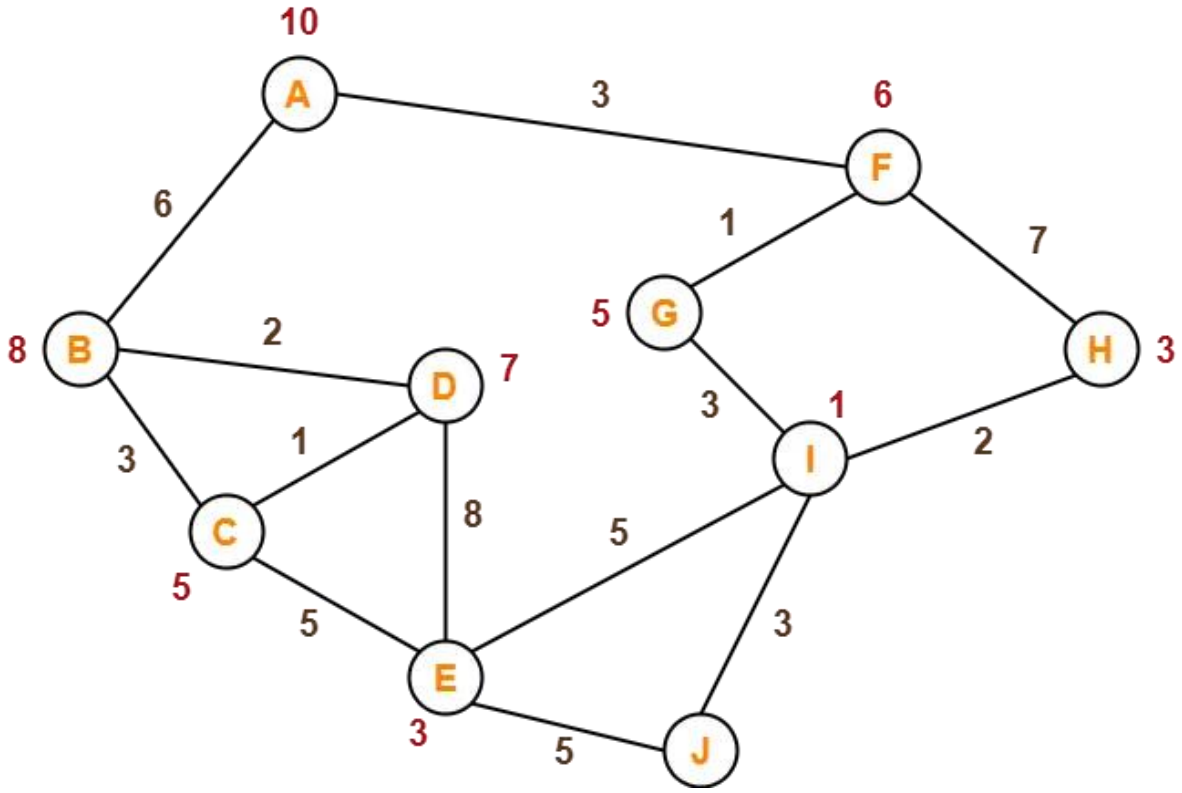
**Your Task:**

1. Model this problem as a **state-space search problem**, representing each state as a tuple (amount_in_Container_A, amount_in_Container_B).
2. Identify the **initial state**, **goal state**, and **successor states** based on the legal operations.
3. Implement a search algorithm (such as Greedy Best-First Search, or A* Search) to find a sequence of operations that achieves the goal.
4. Display the **complete path** from the initial state (0, 0) to the goal state (2, _), including all intermediate states and the actions taken.
5. Discuss whether your chosen search method guarantees the **optimal solution**, and justify your reasoning.

**Example:**

- Initial State: (0, 0)
- Goal State: (2, _) (Container A contains exactly 2 liters, Container B can contain any amount)

**Q5.** Consider the following graph:




The numbers written on edges represent the distance between the nodes. The numbers written on nodes represent the heuristic value. Find the most cost-effective path to reach from start state A to final state J using A* Algorithm.

**Q6.** Given a set of cities and the distances between every pair of cities, implement a **Greedy Best-First Search** algorithm to find a route from a starting city to a destination city. The heuristic values for all cities are provided in the table below. In this search, the evaluation function is defined as:

$$f(n) = h(n)$$

where $h(n)$is the heuristic value estimating the distance from city $n$to the goal city.

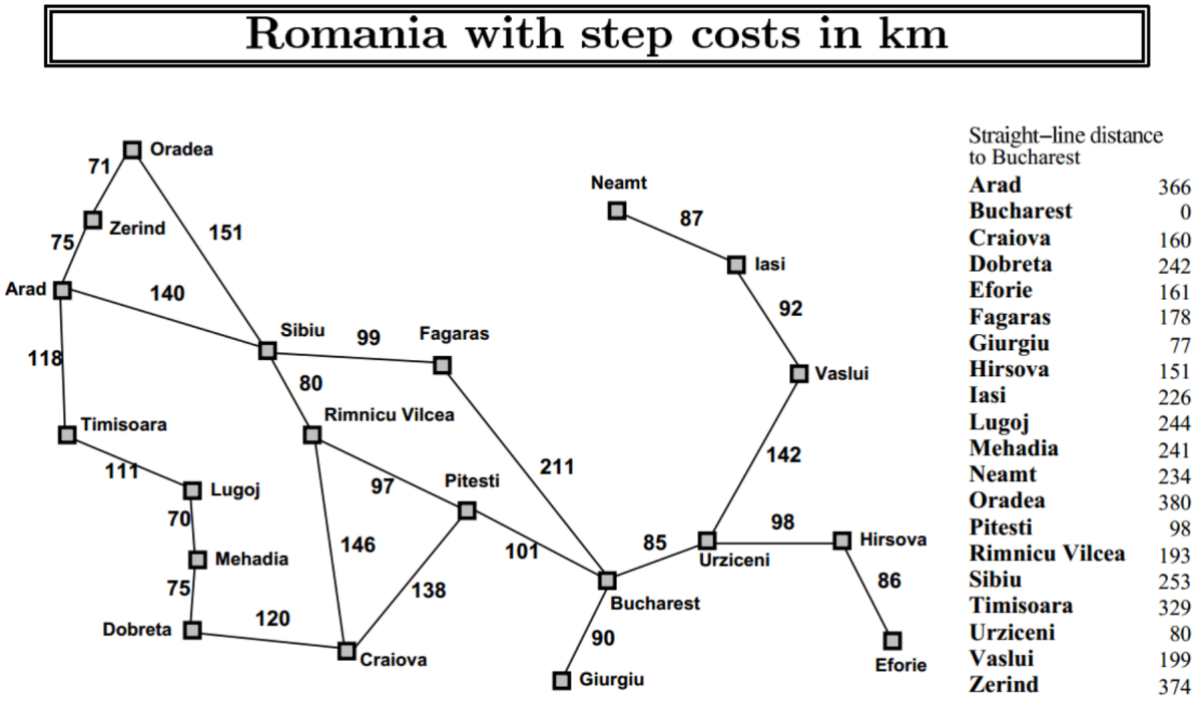
The problem requires finding a route from the starting city **Arad** to the destination city **Bucharest**, using the Greedy Best-First Search strategy. Your program should:

1. Represent the cities and their connections appropriately (e.g., as a graph).
2. Use the given heuristic values to guide the search.
3. Display the sequence of cities visited and the route selected by the algorithm.
4. Calculate the total distance traveled along the route.
5. Discuss whether Greedy Best-First Search guarantees the shortest path in this scenario.

**Assumptions:**

- All distances between cities are known.

- The heuristic values provided are an estimate of the straight-line distance to Bucharest.




**Q7.** In a modern smart city, a new airline wants to offer passengers the **most direct route** between cities. Each city is connected to some other cities by direct flights, and the cost of each flight represents the travel distance in kilometers. Additionally, the airline has coordinates for each city to estimate the straight-line distance to the destination. This estimate will be used as a **heuristic** to guide the search for the fastest route.

Your task is to implement a program to **find the most direct route from a starting city to a destination city** using **Greedy Best-First Search**. The program should use the Euclidean distance between cities as a heuristic to select which city to explore next.

**Input:**

1. **Flights Dictionary:** A mapping where each key is a departure city, and its value is a dictionary of directly connected cities with their flight distances.
2. **City Coordinates:** Coordinates of each city used to compute Euclidean distances.
3. **Start City:** The origin of the flight.
4. **Destination City:** The goal city for the flight.

**HeuristicFunction:**

The heuristic $h(n)$is the Euclidean distance from the current city to the destination city:

$$h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$

**Output:**

- The **sequence of cities** representing the path chosen by the algorithm.
- The **total distance** traveled along this path.

**Algorithm Requirements:**

1. Initialize the search from the **start city** with zero cost.
2. Use a **priority queue** to always expand the city with the lowest heuristic value.
3. Generate successor cities only if they have **not been visited** to prevent cycles.
4. Terminate when the destination city is reached or when no more cities are left to explore.
5. Display the **path taken**, the **sequence of nodes expanded**, and the **total travel distance**.

```
city_flights = {
    'New York': {'Philadelphia': 80, 'Boston': 215},
    'Philadelphia': {'New York': 80, 'Washington D.C.': 135},
    'Boston': {'New York': 215, 'Providence': 50},
    'Washington D.C.': {'Philadelphia': 135, 'Richmond': 95},
    'Providence': {'Boston': 50}
}
city_coords = {
    'New York': (40.7128, -74.0060),
    'Philadelphia': (39.9526, -75.1652),
    'Boston': (42.3601, -71.0589),
    'Washington D.C.': (38.9072, -77.0369),
    'Providence': (41.824, -71.4128),
    'Richmond': (37.5407, -77.4360)
}
start_city = "New York"
goal_city="Washington D.C."
```

**Q8.** A factory has a robot tasked with collecting parts from a workshop and delivering them to an assembly station. The workshop floor is represented as a **5×5 grid**, where each cell may contain a part to be collected. The robot starts at the top-left corner `(0, 0)` and must reach the delivery station at the bottom-right corner `(4, 4)`.

The robot can move in four directions: **Up, Down, Left, Right**, and each move has an associated traversal cost that may vary depending on obstacles, floor conditions, or other factors. The goal is for the robot to **collect the required parts and reach the destination efficiently**, prioritizing paths that appear closest to the goal.

Implement a **Greedy Best-First Search** algorithm to guide the robot from the start position to the destination while collecting parts. Use a **heuristic function** based on the **Manhattan distance** from the current position to the destination to determine which grid cell the robot should explore next.

**Input:**

1. **Grid:** A 5×5 grid representing the workshop floor, where each cell contains:
   - `0` for empty space
   - `1` for a cell containing a part
2. **Traversal Cost:** The cost of moving between adjacent cells (can vary per move).
3. **Start Position:** `(0, 0)`
4. **Destination:** `(4, 4)`

**Heuristic Function:**
Use the Manhattan distance as a heuristic:

$$h(n) = | x_{current} - x_{goal} | + | y_{current} - y_{goal} |$$

**Output:**

- The **sequence of grid positions** the robot moves through.
- The **order in which parts are collected**.
- The **total traversal cost** along the chosen path.

**Requirements:**

1. **Implement Greedy Best-First Search,** always expanding the node with the lowest heuristic value.
2. Avoid revisiting nodes unnecessarily to prevent cycles.
3. Display the **path**, the **collected parts**, and the **total cost**.

| R | 0 | 0 | 1 | 0 |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | D |

```
Path: (0,0) → (0,1) → (0,2) → (0,3) → (1,3) → (1,4) → (2,4) → (3,4) → (3,3)  → (4,4)
Collected Parts at: (0,3), (3,3)
```

One possible solution is this.

**Q9.** Apply the Greedy Best First Search to find the solution path from S to G. Write all steps as well as open and closed lists.

Given nodes and their heuristic values:

- S(h=7), A(h=9), B(h=4), C(h=2), D(h=5), E(h=3), G(h=0)

Graph edges:

- S -> A, S -> D
- A -> B

- B -> C, B -> E
- C -> G
- D -> E
- E -> G